

APIデータを可視化する

1年分の閲覧数の謎を解く

rkskmt

1

前回のあらすじ

2

「なにかがおかしい」ところまで来た

- 前回： 記事「七夕」の 2025年まるごと1年分 の日別アクセス数を取得した

Code

```
1 import requests
2 import numpy as np
3
4 url = ("https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
5        "ja.wikipedia/all-access/user/"
6        "七夕/daily/20250101/20251231")
7 headers = {"User-Agent": "example.com/lesson"}
8
9 response = requests.get(url, headers=headers)
10 data = response.json()
11
12 views = np.array([item["views"] for item in data["items"]])
```

統計量	値
最大	47,656回
最小	58回
平均	467.6回
中央値	154回

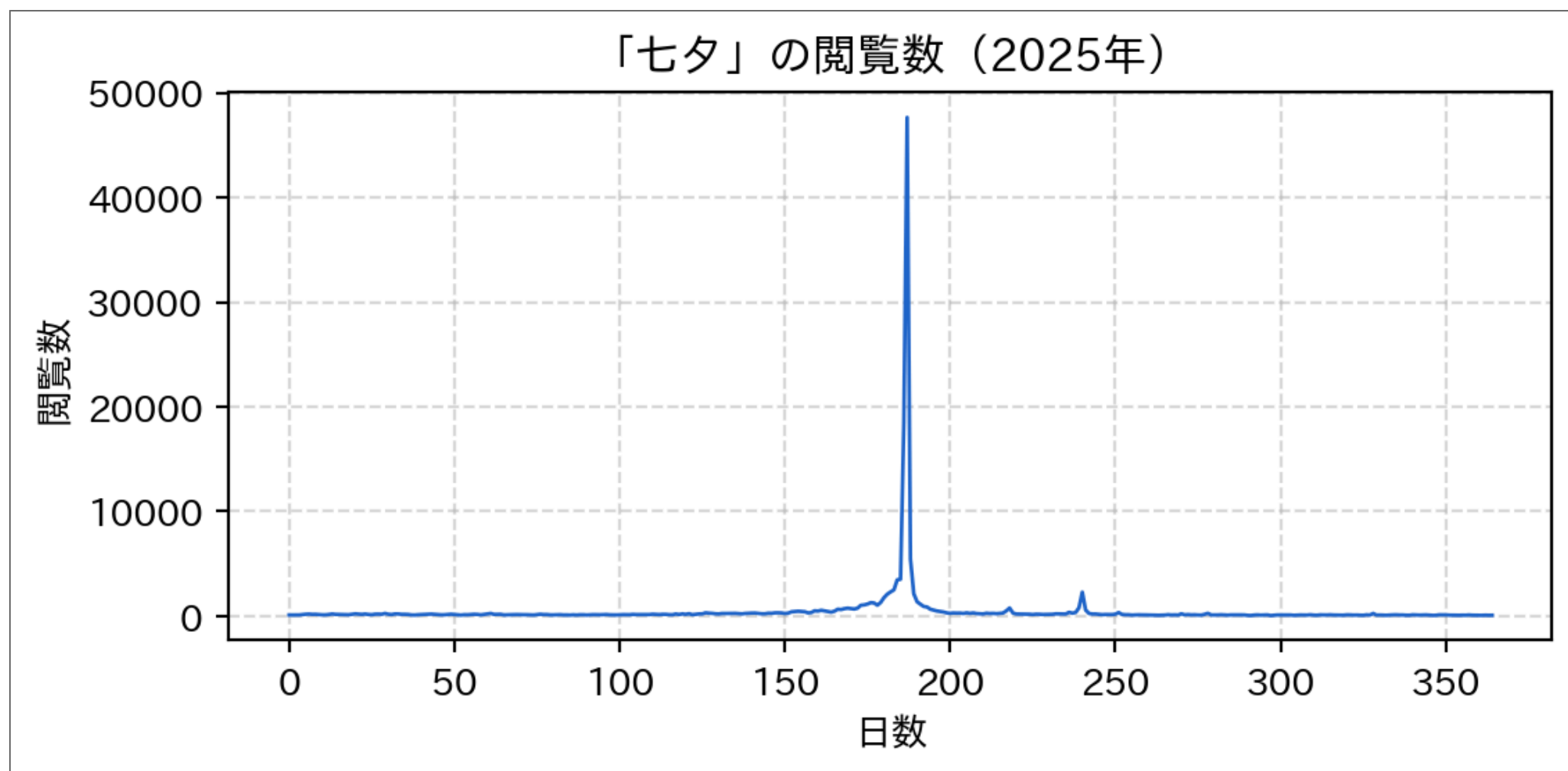
- **最大が中央値の約300倍**。平均と中央値も大きくズレている
- 統計量だけでは、これ以上なにも分からない — では、どうする？

3



まずは可視化する

- 数値だけでは見落とすので **まず描く** — [データの可視化](#) の大原則
- 同じURL/ヘッダーで取得したあと **`plt.plot(views)`** するだけ



▶ コードを見る

① 1日だけ突出している

- ほぼ平坦な中、**1日だけ大きな山** — 数値だけ見ていると気づけない（アンスコムの教訓と同じ）
- そして見覚えがあるはず： [データの可視化の冒頭で見た 謎の図](#) は、実はこのデータ

山の正体：「いつ」を特定する

- 山の **高さ** は `views.max()`。知りたいのは **「いつ」**
- `item["timestamp"]` も同じ内包表記で取り出すと、**views** と **同じインデックスで並ぶ** 日付の配列ができる
- `np.argmax()` は最大値の **インデックス** を返す（値ではない点に注意） — そのインデックスで **dates** を引く

Code

```
1 dates = np.array([item["timestamp"] for item in data["items"]])
2
3 max_i = np.argmax(views)
4 print(f"{dates[max_i]} に {views[max_i]} 回")
```

📢 クイズ

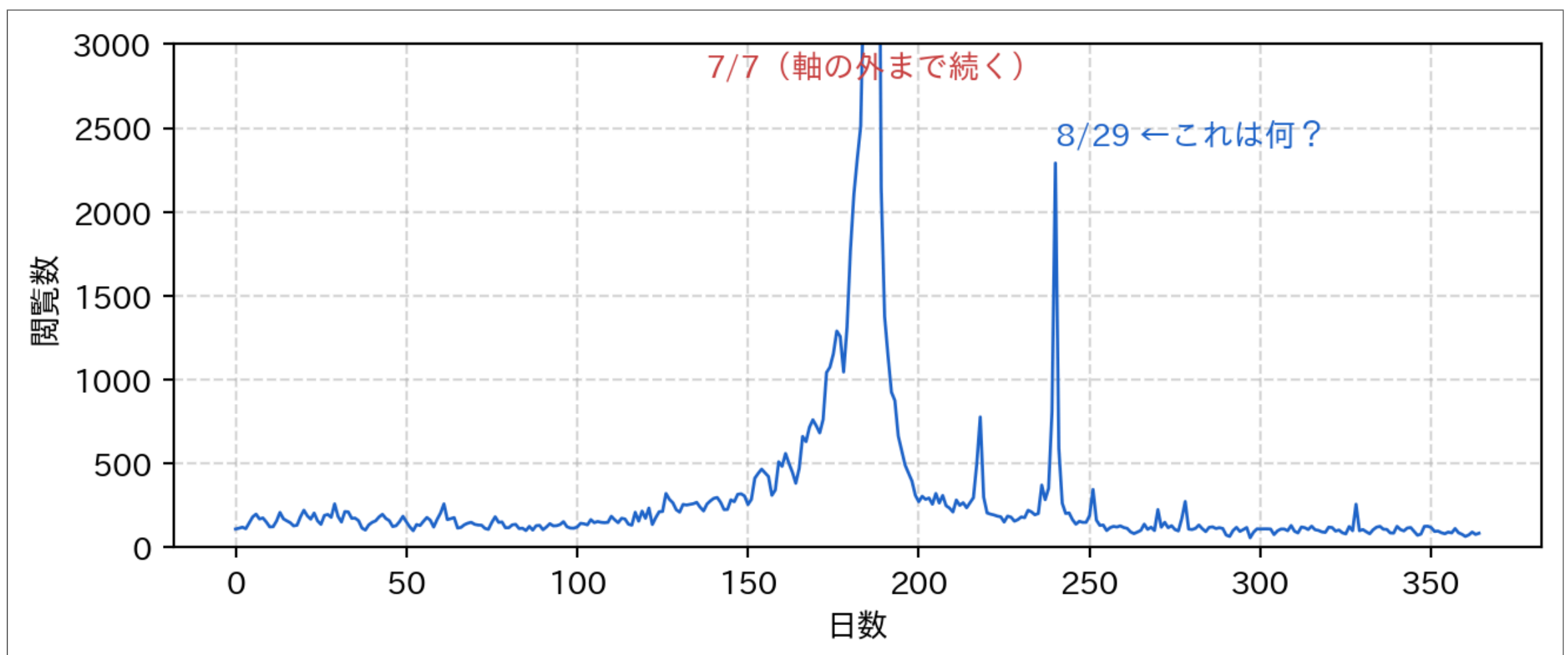
記事は「七夕」。47,656回も読まれた**その1日**は、何月何日だと思う？

1 2025070700 に 47656 回

- **2025年7月7日** — 七夕の当日
- みんな、**行事の当日にその行事を調べる**
 - 「七夕って何の日？」「短冊はなぜ？」 — 検索が記事に殺到する
- 1年分の人間の行動が、API経由の数列ひとつに写っている

5

よく見ると、もう一つ山がある

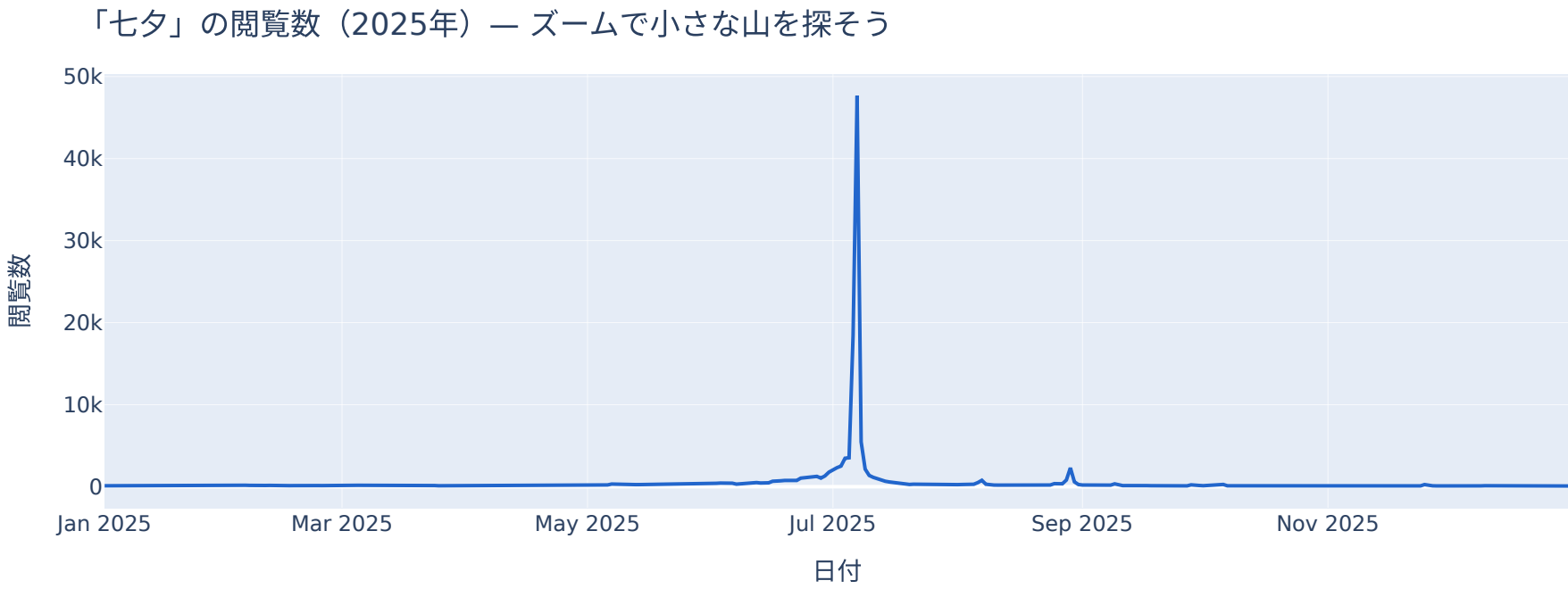


- y軸を拡大すると、**8月29日にも小さな山**（2,291回） — 七夕でもないのに、なぜ？
- 自分のグラフでも `plt.ylim(0, 3000)` を足せば、同じ「足元の拡大」ができる

6



- 小さな山の正体は？ **グラフをドラッグで拡大**して確かめよう



- 正体： **伝統的七夕** — 旧暦7月7日にあたる日（2025年は8月29日）
→ 国立天文台が毎年発表している。7月7日は梅雨だが、**旧暦の七夕は夏空で天の川が見える**
- **ズームしないと見えない発見もある** — 軸の範囲は「正直さ」だけでなく「発見」の道具でもある

7

まとめ

やりたいこと	コード
まず描く	<code>plt.plot(views)</code>
最大値の位置	<code>max_i = np.argmax(views)</code>
位置から日付	<code>dates[max_i]</code> （同じindexの配列をリンク）
足元を拡大	<code>plt.ylim(0, 3000)</code>

- 統計量で「おかしい」と気づき、**図で正体を突き止める**
- 大原則 *Always plot your data first.* は **API由来のデータ** にもそのまま効く

8

9

演習：1枚のグラフに2本の折れ線



前回の演習で取得した `views_desktop` と `views_mobile_web` を、同じグラフに重ねて比較せよ。

1. `plt.plot()` を2回呼ぶ — それだけで重なる
2. グラフの **タイトルに max / min / mean** を表示

⚠ やってはいけない

- 軸を揃えず2枚並べる → 比較しづらい
- 軸の範囲を片方だけ狭める → 印象操作 ([matplotlib入門](#) の「軸の範囲で印象を操る」でやった罠)

▶ 解答例を見る

10

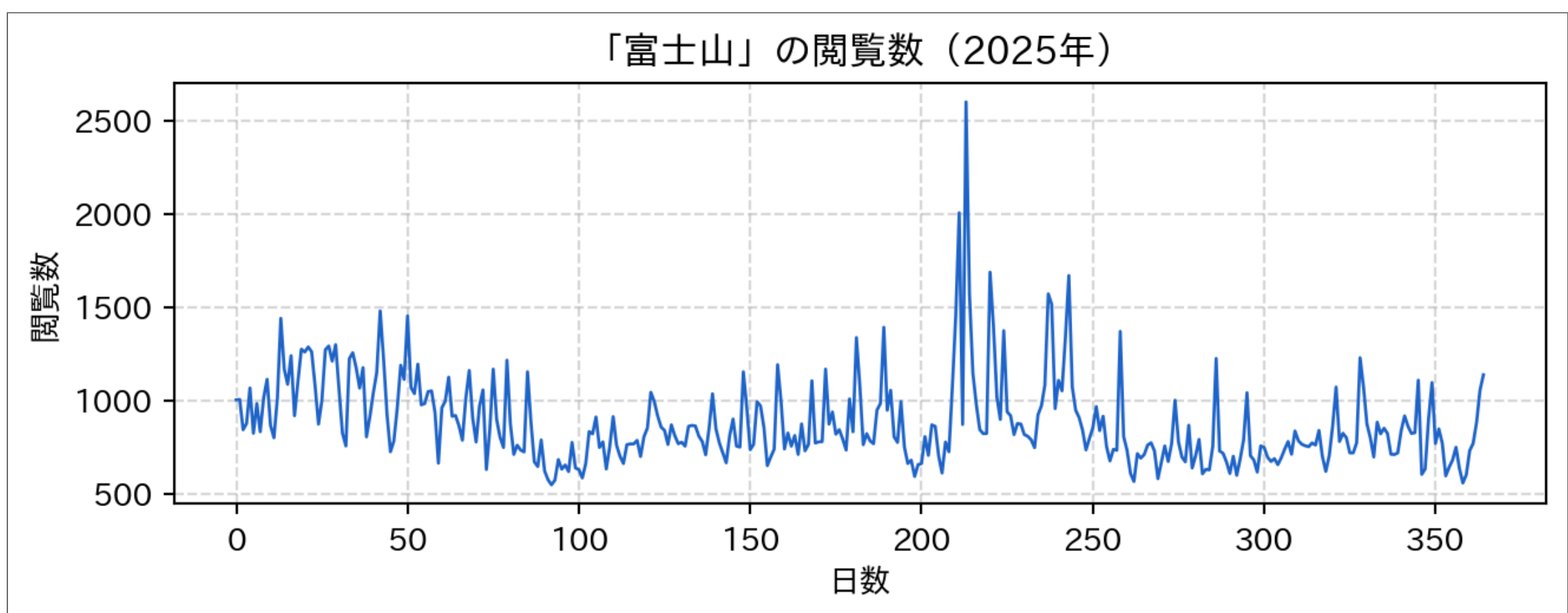
演習：富士山には山があるか



「七夕」のコードの **記事名を 富士山 に変えるだけ** で2025年1年分を取得して描き、次の問いに答えよ。

- 問い：七夕のような **鋭い山（スパイク）** はあるか？
- 問い：スパイクがないなら、**何が見えるか？** 自分の言葉で説明せよ

▶ 解答例を見る



① 種明かし：針はない。でも「波」がある

- 七夕のような数百倍のスパイクは **ない**（最大でも中央値の約3倍）
- 代わりに **8月（登山シーズン）と1～2月** が高い、ゆるやかな波
→ 夏が高いのは分かる。冬はなぜ高い？ — 考えてみよう
- **スパイクだけが発見ではない**。波（季節性）も、データが教えてくれる人間の行動

11

ここまでの流れ

- 前回：**requests** で **取得**、NumPy で **集計** — 「なにかがおかしい」
- 今回：matplotlib で **可視化** — 山の正体（7/7と8/29）を突き止めた
- 統計量 → 違和感 → 図 → 発見、という **データ分析の基本サイクル** を一周した

12

参考リンク

- [Wikimedia REST API](#) 
- [Pageviews API ドキュメント](#) 
- [国立天文台「伝統的七夕」](#) 

13